Équations de récurrences

Andrey Martinez Cruz

Complexité d'algorithmes itératifs

Quelle est la complexité de cet algorithme :

```
def quelque_chose(M, i):
    if i >= len(M):
        return False
    j = 1
    ok = True
    while j < len(M[i]):
        if M[j - 1] > M[j]:
            ok = False
        else:
            j = j + 1
    return ok or quelque_chose(M, i + 1)
```

Quelle est la complexité temporelle de cet algorithme ?

Complexité d'algorithmes itératifs

Quelle est la complexité de cet algorithme :

```
def quelque_chose(M, i):
    if i >= len(M):
        return False
    j = 1
    ok = True
    while j < len(M[i]):
        if M[j - 1] > M[j]:
            ok = False
        else:
            j = j + 1
    return ok or quelque_chose(M, i + 1)
```

Quelle est la complexité temporelle de cet algorithme ? $\Theta(nm)$

Complexité d'algorithmes itératifs

Quelle est la complexité de cet algorithme :

```
def quelque_chose(M, i):
    if i >= len(M):
        return False
    j = 1
    ok = True
    while j < len(M[i]):
        if M[j - 1] > M[j]:
            ok = False
        else:
            j = j + 1
    return ok or quelque_chose(M, i + 1)
```

Quelle est la complexité temporelle de cet algorithme ? $\Theta(nm)$

Comment formuler la complexité asymptotique ?

Complexité d'algorithmes itératifs

Quelle est la complexité de cet algorithme :

```
def quelque_chose(M, i):
    if i >= len(M):
        return False
    j = 1
    ok = True
    while j < len(M[i]):
        if M[j - 1] > M[j]:
            ok = False
        else:
            j = j + 1
    return ok or quelque_chose(M, i + 1)
```

Quelle est la complexité temporelle de cet algorithme ? $\Theta(nm)$

Comment formuler la complexité asymptotique ?
Sous forme d'équation de récurrence aux diférences finies :

$$T(n) = T(n-1) + m$$

avec comme cas de base $T(0) = 1$.

Complexité d'algorithmes itératifs

Quelle est la complexité de cet algorithme :

```
def quelque_chose(M, i):
    if i >= len(M):
        return False
    j = 1
    ok = True
    while j < len(M[i]):
        if M[j - 1] > M[j]:
            ok = False
        else:
            j = j + 1
    return ok or quelque_chose(M, i + 1)
```

Quelle est la complexité temporelle de cet algorithme ? $\Theta(nm)$

Comment formuler la complexité asymptotique ?
Sous forme d'équation de récurrence aux diférences finies :

$$T(n) = T(n-1) + m$$

avec comme cas de base $T(0) = 1$.
Note : par simplicité, on assume que pour tout itérations dans la boucle, il y m itérations.

Équations de récurrences

Équations de récurrences : fonction dont la définition consiste à s'appelait elle-même. Voici quelques exemples d'équations de récurrences :

- $T(n) = 2T(n-1) + n$
- $T(n) = T(n-2) + n \log n$
- $T(n) = T(n-1) + T(n-4) + n$
- $T(n) = T(\frac{n}{4}) + n^2$
- $T(n) = 4T(\frac{n}{6}) + n \log n$
- $T(n) = \sqrt{n}T(\sqrt{n}) + 1$

Équations de récurrences

Équations de récurrences : fonction dont la définition consiste à s'appelait elle-même. Voici quelques exemples d'équations de récurrences :

- $T(n) = 2T(n-1) + n$
- $T(n) = T(n-2) + n \log n$
- $T(n) = T(n-1) + T(n-4) + n$
- $T(n) = T(\frac{n}{4}) + n^2$
- $T(n) = 4T(\frac{n}{6}) + n \log n$
- $T(n) = \sqrt{n}T(\sqrt{n}) + 1$

Note : pour ce cours, on s'intéressera simplement à une petite partie de ces équations : les équations de récurrences aux différences finies et aux divisions finies et à quelque cas spécial.

Dans le cadre d'INF5130

On verra notamment les équations de récurrences aux différences finies, aux divisions finies et quelques cas spéciaux.

Pour aujourd'hui, on s'intéressera seulement sur les techniques de résolutions s'appliquant aux équations de différences finies. On verra la prochaine fois les autres. ;)

Techniques de résolutions

Techniques

Il existe plusieurs techniques permettant de donner à une équations de récurrences une complexité asymptotique proche ou exacte.
Voici les principales qu'on abordera très fort probablement :

- «Substitution par déroulement»
- Méthode en arbre
- Utilisation du théorème général (master theorem).

Équations de récurrence aux différences finies

Forme

Une équation de récurrence aux différences finies prend la forme suivante :

$$T(n) = \alpha T(n - \beta) + f(n)$$

où $\alpha, \beta > 0, \in \mathbb{N}$ et $f : \mathbb{N} \to \mathbb{R}^+$. Il s'agit de la forme de base qu'on abordera dans ce cours.

Anatomie

$$T(n) = \alpha T(n - \beta) + f(n)$$

- α : nombre d'appels récursifs effectués
- β : constante de réduction du problème
- f : fonction de travaille après les appels récursifs

Résolution de déroulement

Principe : prendre certaines valeurs et essayer de trouver une formule équivalent et qui est non récursif. Pour être certain que la formulation soit équivalente, il faudrait démontrer par induction que la substitution est valide.

Par exemple, avec l'équation de récurrence de départ, on a $m = 3, n = 3$

$$\begin{aligned} T(3) &= T(2) + 3 \\ &= (T(1) + 3) + 3 \\ &= ((T(0) + 3) + 3) + 3 \\ &= ((1 + 3) + 3) + 3) \\ &= 10 \end{aligned}$$

Résolution de déroulement

Avec un n et m quelqueconque

$$
\begin{aligned}
T(n) &= T(n) + m \\
&= (T(n-1) + m) + m \\
&= ((T(n-2) + m) + m) + m \\
&= \vdots \\
&= T(n-i) + m + \ldots + m \\
&= \vdots \\
&= T(0) + m + \ldots + m
\end{aligned}
$$

Ça a l'air familier ...

Résolution de déroulement

Avec un n et m quelqueconque

$$\begin{aligned} T(n) &= T(n) + m \\ &= (T(n-1) + m) + m \\ &= ((T(n-2) + m) + m) + m \\ &= \vdots \\ &= T(n-i) + m + \ldots + m \\ &= \vdots \\ &= T(0) + m + \ldots + m \end{aligned}$$

Ça a l'air familier ...

On peut poser que

$$T(n) = \sum_{i=1}^{n}(m) + 1$$

En simplifiant, on a $T(n) = nm + 1 \in \Theta(nm)$

À vous de jouer !

Quelle est la complexité asymptotique de l'équation de récurrence suivante

$$T(n) = T(n-1) + n^2$$

À vous de jouer !

Quelle est la complexité asymptotique de l'équation de récurrence suivante

$$T(n) = T(n-1) + n^2$$

$T(n) \in \Theta(n^3)$